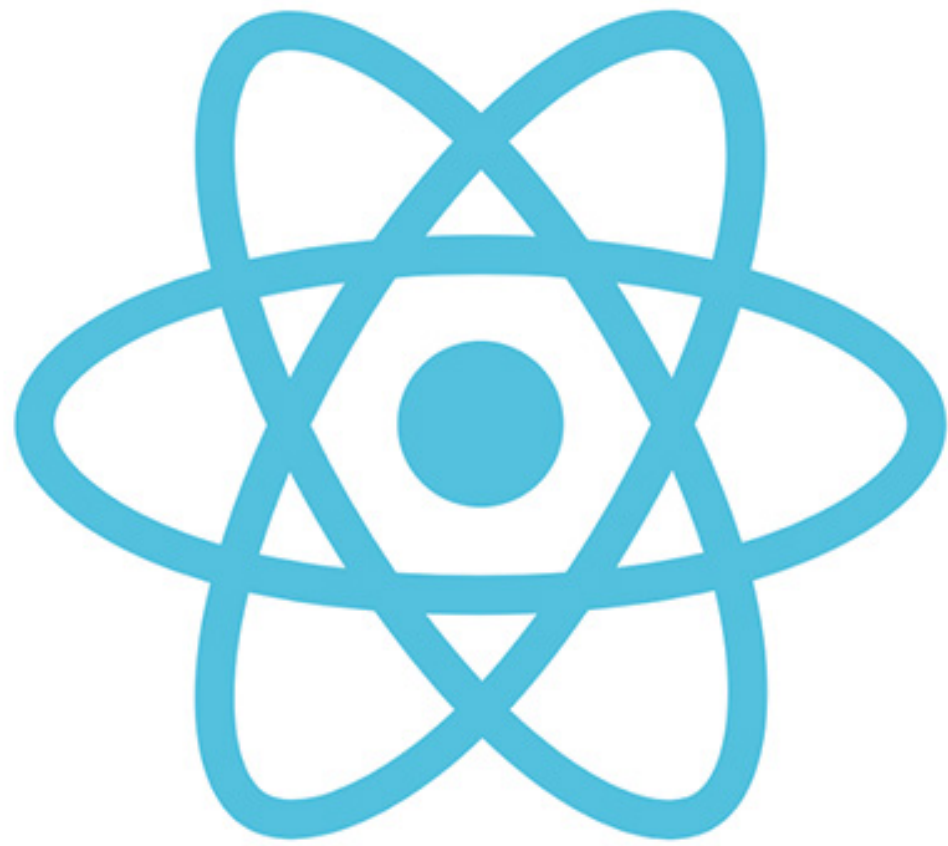




React JS

웹 프론트의 개발 흐름

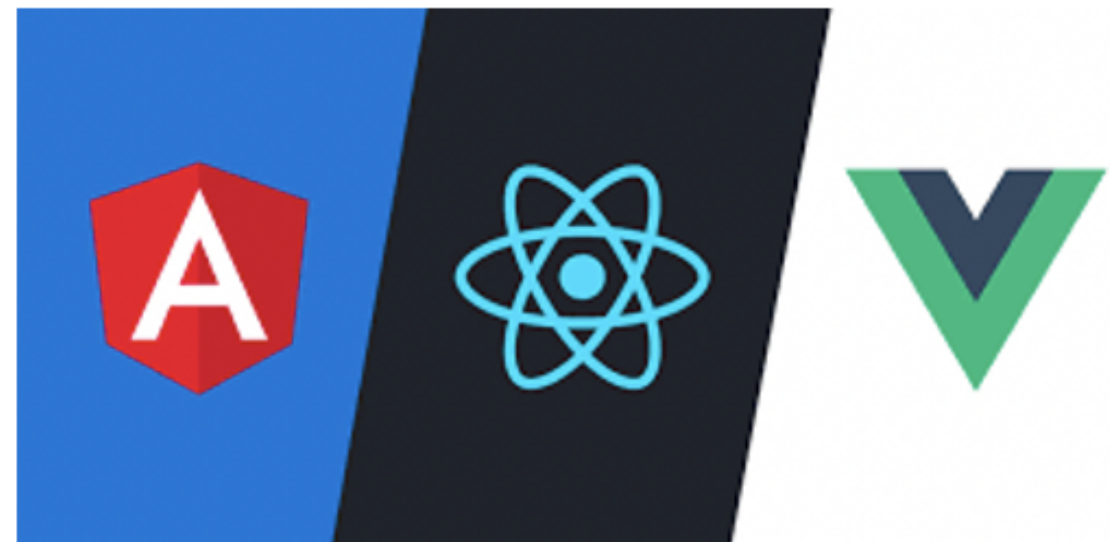
웹 기능이 복잡해지면서 바닐라js, jquery 등의 한계를 극복해주는 프레임워크나 라이브러리가 등장



바닐라JS

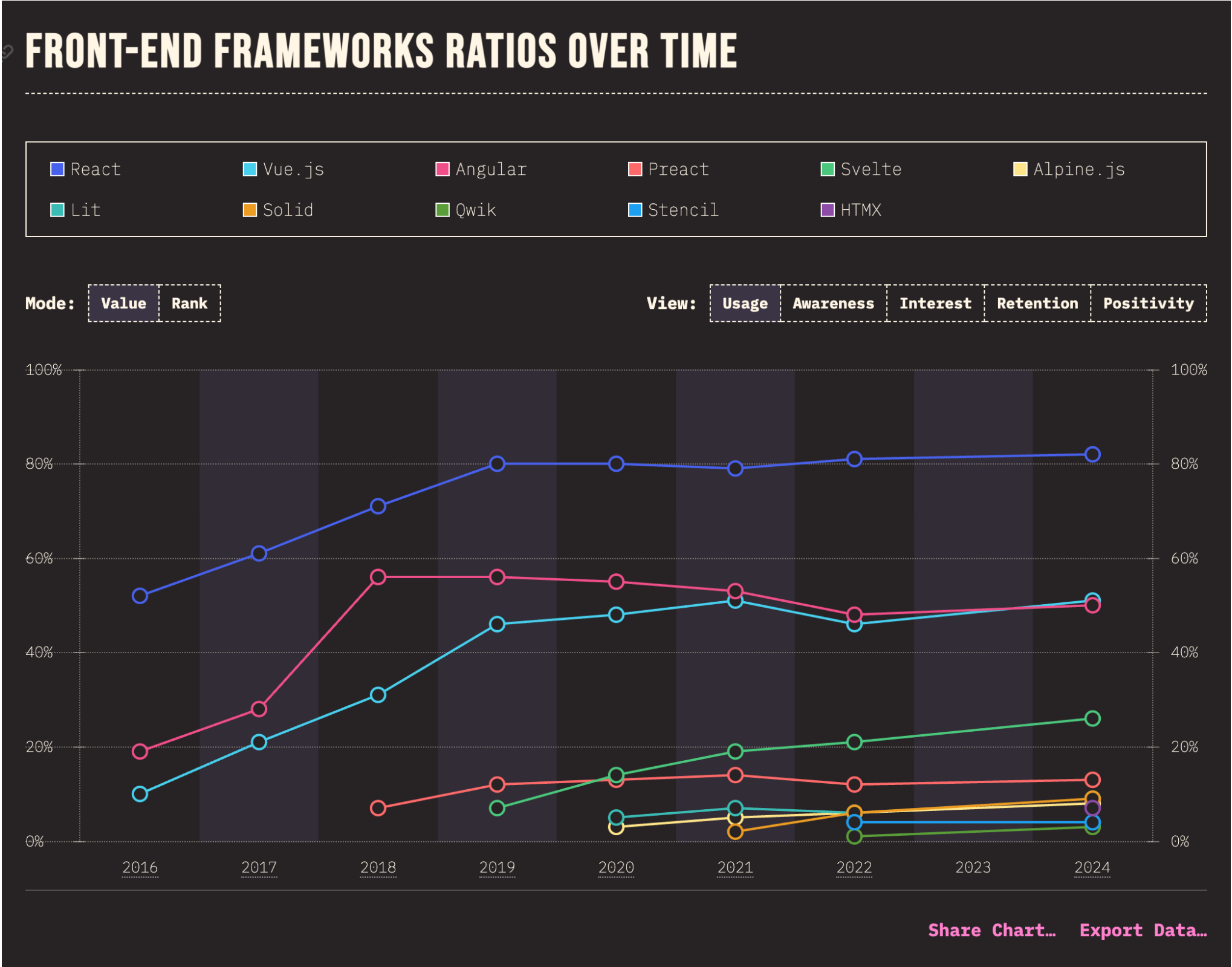


제이쿼리



Angular, React, Vue

주요 라이브러리 점유율



React 란

- React는 Meta(구 Facebook)에서 개발한 JavaScript 라이브러리
- 사용자 인터페이스(UI) 구축을 위해 사용
- 컴포넌트 기반으로 모듈화된 UI를 쉽게 설계
- 한 번 배워서 어디서나 사용 가능 (웹, 모바일, 데스크탑 등)



React 장점

- 컴포넌트 재사용성
 - 동일한 코드를 여러 곳에서 재사용 가능
- 빠른 성능
 - 가상 DOM(Virtual DOM)을 사용해 빠르고 효율적인 업데이트
- 커뮤니티와 생태계
 - 전 세계적으로 활발한 개발자 커뮤니티와 방대한 라이브러리 및 도구
- 단일 방향 데이터 흐름
 - 데이터가 예측 가능한 방식으로 애플리케이션을 통해 흐름



- npm(Node Package Manager)은 JavaScript 및 세계 최대의 소프트웨어 레지스트리 패키지 관리자
- npm registry에는 Node.js에서 사용되는 각종 패키지들이 존재
- npm을 사용해서 필요한 라이브러리를 설치
- Package.json, Package-lock.json으로 의존성 관리
- npm init 으로 초기화
- npx를 사용하면 패키지를 설치하지 않고 바로 실행

Package.json

항목	내용
package name	패키지 이름으로 프로젝트 혹은 현재 애플리케이션 이름을 말한다.
version	패키지, 프로젝트, 애플리케이션 버전 면을 말한다.
description	패키지, 프로젝트, 애플리케이션의 설명을 말한다.
entry point	프로젝트에서 가장 먼저 실행되는 자바스크립트 실행파일로 웹 서버의 경우 index.js 혹은 app.js 로 지정해 준다.
test command	코드 테스트 시에 입력해 줄 명령어로, 사용하지 않을 경우 빈 값으로 지정된다.
git repository	코드를 저장해둔 깃(Git) 저장소 주소로 사용하지 않을 경우 빈 값으로 지정된다.
keywords	패키지, 프로젝트, 애플리케이션의 검색을 위한 키워드
license	패키지, 프로젝트, 애플리케이션의 라이선스로 기본값은 ISC (Internert Systems Consortium)에서 허용한 자유 소프트웨어 라이선스

React 프로젝트 생성

npm create vite@latest

```
> npx
> "create-vite"

◇ Project name:
  sample1

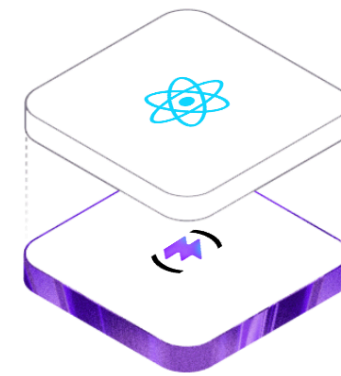
◇ Select a framework:
  React

◇ Select a variant:
  TypeScript

◆ Install with npm and start now?
  ● Yes / ○ No
```

VITE v8.0.9 ready in **3054** ms

```
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```



Get started

Edit `src/App.tsx` and save to test `HMR`

Count is 6

React 프로젝트 생성

프로젝트 구성

탐색기

...

▼ SAMPLE1

> node_modules

> public

> src

🔍 .gitignore

🔍 eslint.config.js

<> index.html

{} package-lock.json

{} package.json

📄 README.md

{} tsconfig.app.json

TS tsconfig.json

{} tsconfig.node.json

⚡ vite.config.ts

{} package.json ✕

{} package.json > ...

1 {

2 "name": "sample1",

3 "private": true,

4 "version": "0.0.0",

5 "type": "module",

▷ 디버그

6 "scripts": {

7 "dev": "vite",

8 "build": "tsc -b && vite build",

9 "lint": "eslint .",

10 "preview": "vite preview"

11 },

12 "dependencies": {

13 "react": "^19.2.5",

14 "react-dom": "^19.2.5"

15 },

16 > "devDependencies": { ...

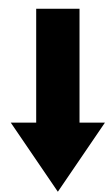
29 }

30 }

JSX

- React에서 사용되는 Javascript 확장 문법
- JSX는 Javascript and XML의 약자
- JSX는 Javascript 코드 안에서 HTML과 유사한 구문을 작성
- 브라우저에서 실행되기 전에 JSX는 Javascript로 변환
- 직관적으로 구성 가능

```
const element = <h1>Hello, JSX!</h1>
```



```
const element =  
  React.createElement('h1', null, 'Hello, JSX!');
```

React로 element 만들기

```
const element1 = <h1 className='greeting'>
  'Hello World!'
</h1>
```

JSX 사용

```
const element2 = React.createElement(
  'h1', {className: 'greeting'},
  'Hello World!'
);
```

React method 사용

```
const element3 = {
  type: 'h1',
  props: {
    className: 'greeting',
    Children: 'Hello World!'
  }
}
```

js 객체 형태
최종적으로 모두 이 형태로 변경

JSX 로 element 만들기

```
root.render(  
  <p>Hello World!</p>  
  <p>Hello World!</p>  
);
```

단일 element로 만들어줘야함

```
root.render(  
  <React.Fragment>  
    <p>Hello World!</p>  
    <p>Hello World!</p>  
  </React.Fragment>  
);
```

```
root.render(  
  <div>  
    <p>Hello World!</p>  
    <p>Hello World!</p>  
  </div>  
);
```

```
root.render(  
  <>  
    <p>Hello World!</p>  
    <p>Hello World!</p>  
  </>  
);
```

형식적으로 단일 element로 만들어줌

JSX 로 element 만들기

```
const tag = <img src={image}> //에러발생 '/' 필요
```

```
const tag = <img src={image}/>
```

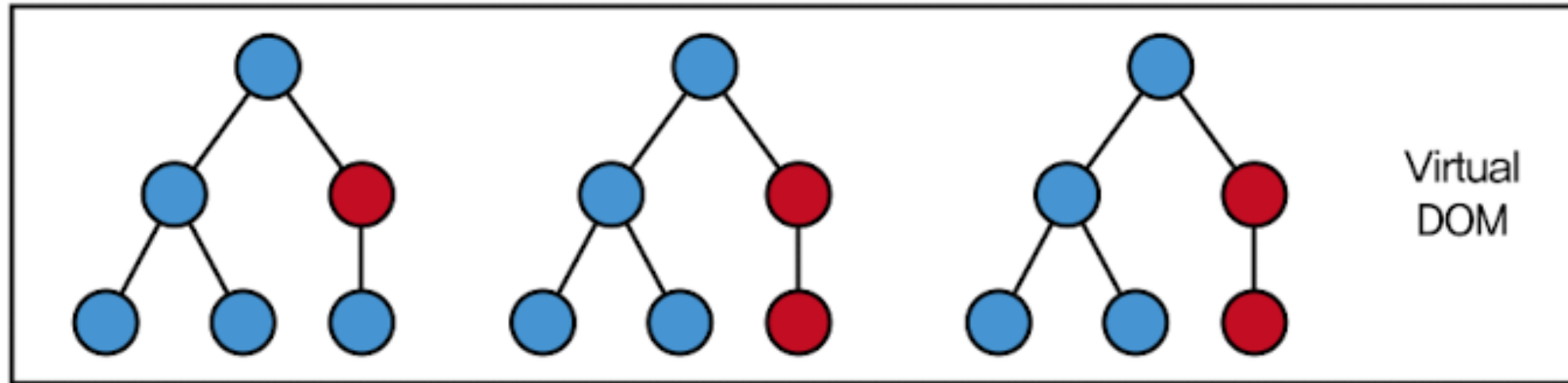
```
const tag = <img src={image}></img>
```

끝 tag가 없는 빈 요소들은 html tag와 달리 끝 tag를 붙여줘야 함

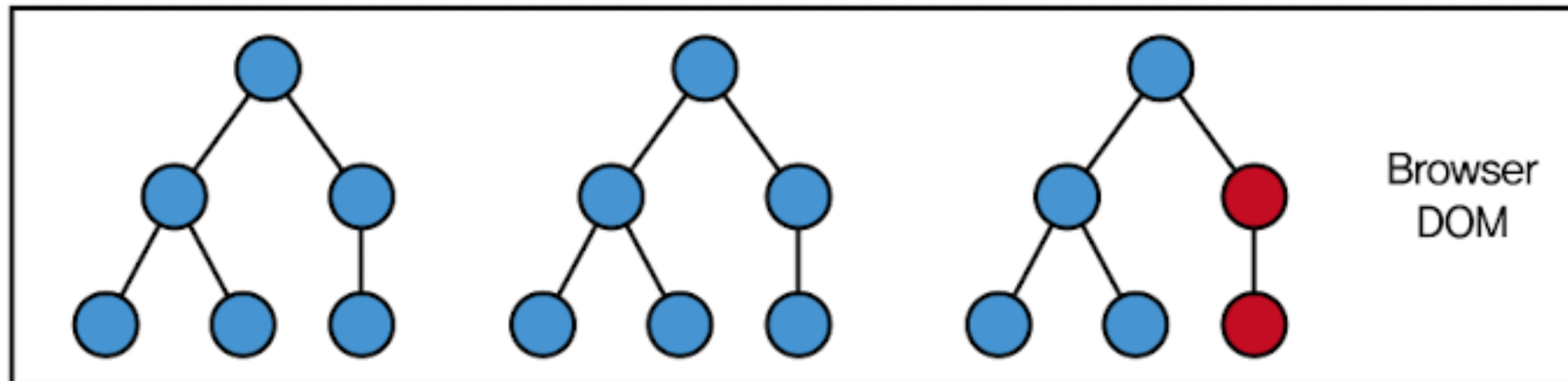
```
const input = (  
  <div className="field">  
    <label htmlFor="title">이름</label>  
    <input type="text" id="title"/>  
  </div>  
);
```

html의 속성 중에 class는 className, for 속성은 htmlFor 라고 해야함
속성중에 두개 이상의 단어로 구성되어있으면 camel case를 사용해야함

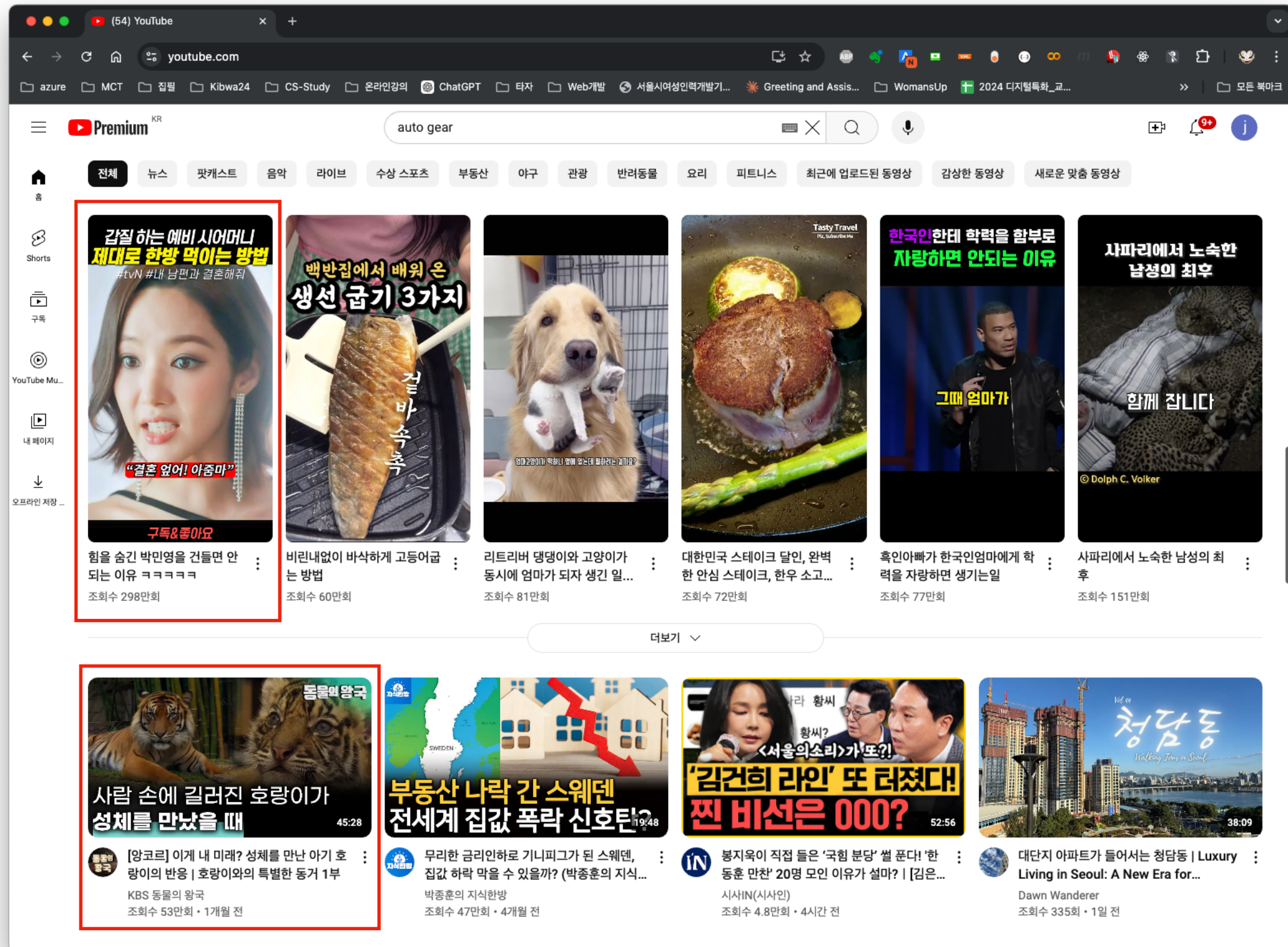
Virtual DOM



State Change → Compute Diff → Re-render



Component



Sample Component - Hello.jsx

html tag와 구분하기 위해서 대문자로 시작

```
export default function Hello() {  
  return (  
    <>  
      <h1>안녕하세요.</h1>  
      <h1>Hello World!</h1>  
    </>  
  );  
}
```

일반적으로 JSX파일은 하나의 컴포넌트만
export 해서 export default를 사용

```
import Hello from './Hello';  
const root = ReactDOM.createRoot(  
  document.getElementById('root'));  
root.render(<Hello />);
```


Props 활용

```
interface WelcomeProps {  
  name: string;  
}
```

```
function Welcome(props: WelcomeProps) {  
  return <h1>안녕, {props.name}</h1>;  
}
```

```
const root =  
  ReactDOM.createRoot(document.getElementById('root'));  
root.render(  
  <React.StrictMode>  
    <Welcome name="손흥민" />  
    <Welcome name="홍길동" />  
  </React.StrictMode>  
);
```

Sample Component - SayHello.jsx

```
interface SayHelloProps {  
  myName: string;  
  to: string;  
}  
  
export default function SayHello(props: SayHelloProps) {  
  return (  
    <>  
      <p>안녕하세요. {props.myName}</p>  
      <p>Hello {props.to}!</p>  
    </>  
  );  
}
```

컴포넌트는 Tag처럼 사용됨
전달해야하는 데이터는 Tag의 속성처럼 전달
{ }에는 javascript의 표현식이 들어감

Sample Component - SayHello.jsx

객체분해 사용

```
export default function SayHello({ myName, to }: SayHelloProps) {  
  return (  
    <>  
      <p>안녕하세요. {myName}</p>  
      <p>Hello {to}!</p>  
    </>  
  );  
}
```

```
import SayHello from './SayHello';  
const root = ReactDOM.createRoot(  
  document.getElementById('root'));  
root.render(<SayHello myName="오이영" to="React" />);
```

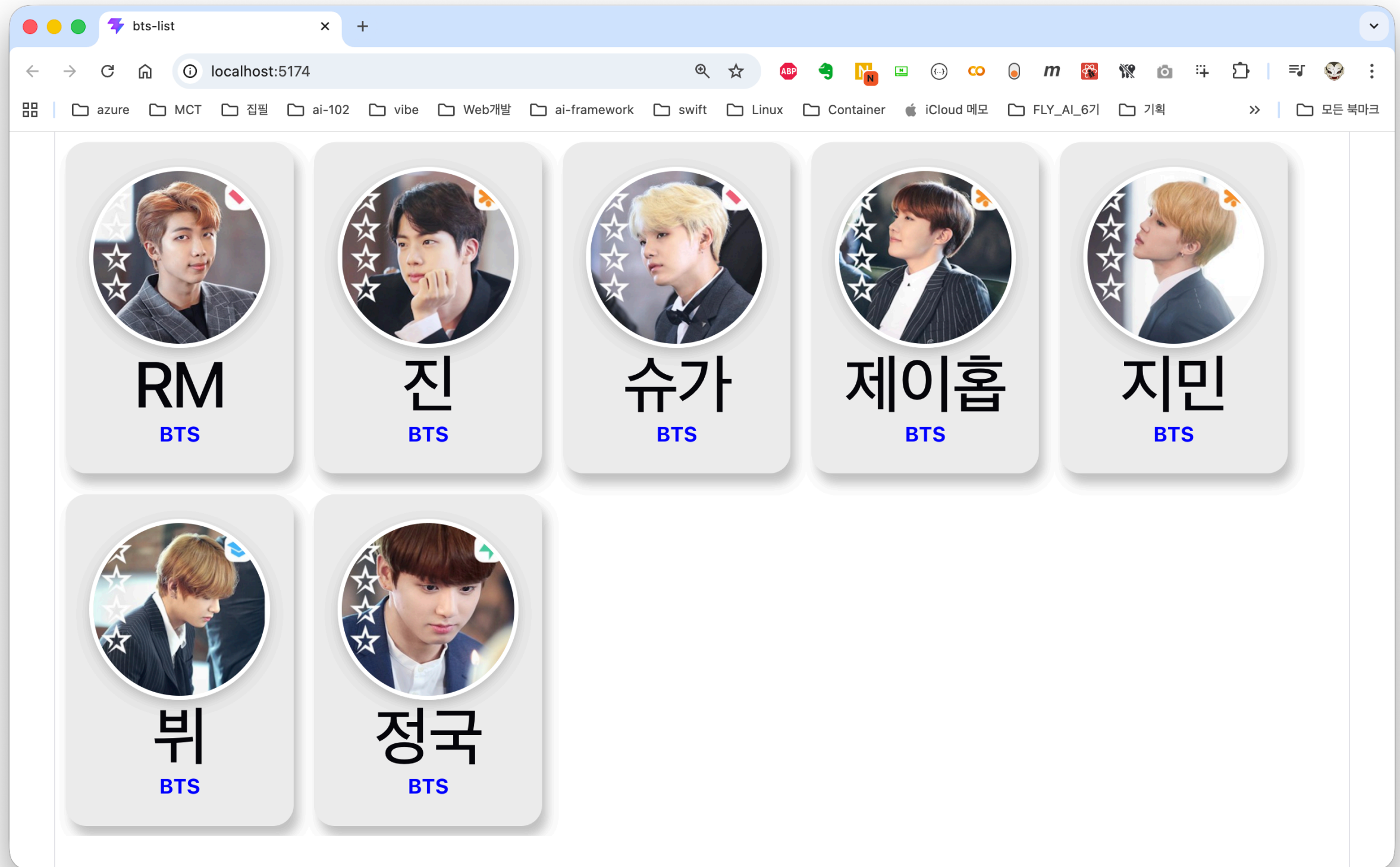
Sample Component - Book.jsx

```
interface BookProps {  
  title: string;  
  author: string;  
}  
  
function Book(props: BookProps) {  
  return (  
    <>  
      <h1>{'이 책의 이름은 ${props.title} 입니다.'}</h1>  
      <h2>{'이 책의 저자는 ${props.author} 입니다.'}</h2>  
    </>  
  );  
}  
  
export default Book;
```

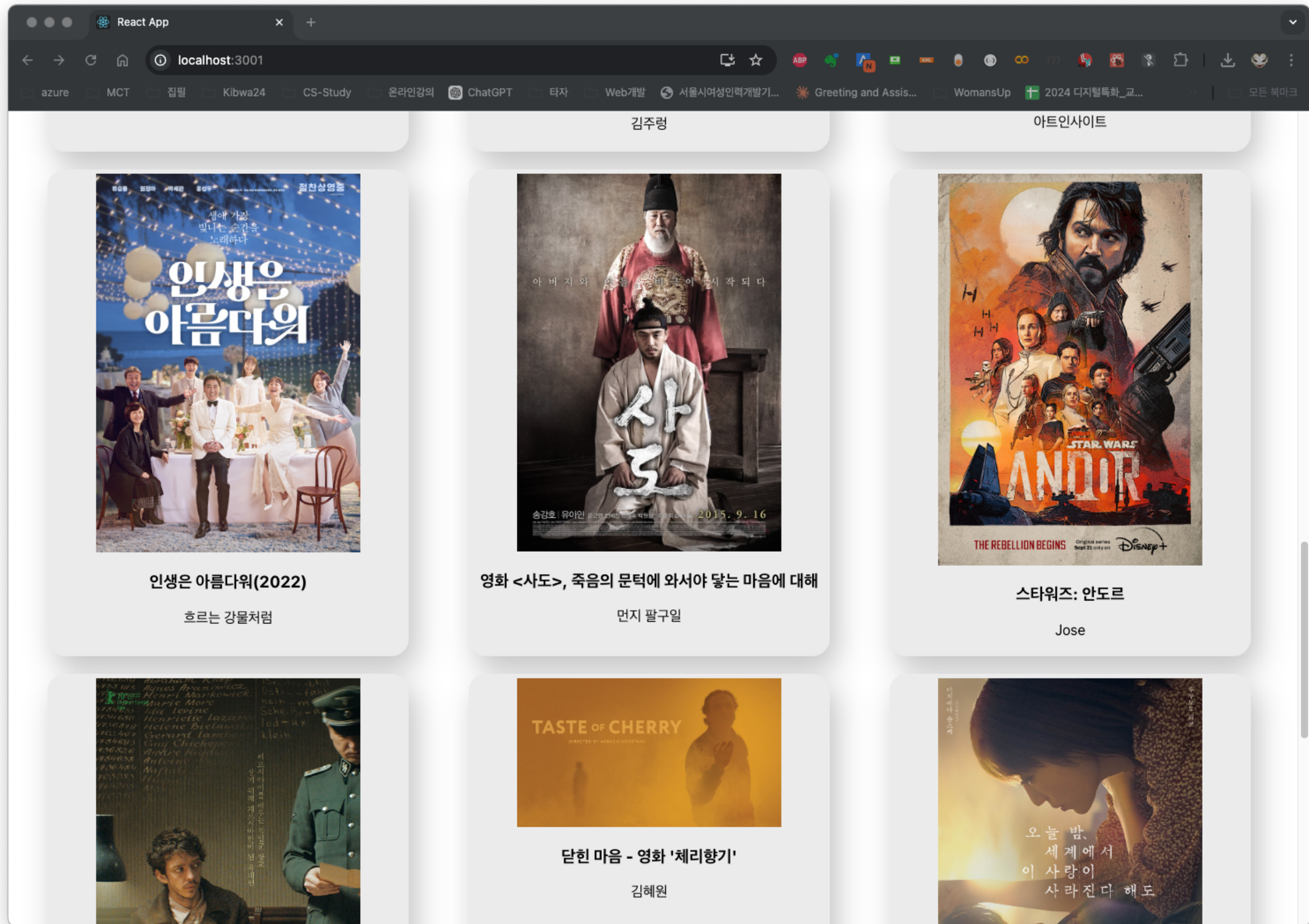
Sample Project - Library.jsx

```
import React from 'react';
import Book from './Book';
function Library() {
  return (
    <>
      <Book title="한입 크기로 잘라먹는 리액트" author="이정환" />
      <Book title="처음 배우는 애저" author="김도균" />
      <Book title="Node.js 교과서" author="조현영" />
    </>
  );
}
export default Library;
```

BTS Component 만들기



영화 Component 만들기



Event와 State

React - Event

- 사용자 입력(클릭, 입력, 마우스 움직임 등)에 응답하는 방식
- HTML과 달리 camelCase로 이벤트 명명
- JSX에서 함수로 이벤트 핸들러 전달

`<button onClick={handleClick}>클릭</button>`

Event 명	설명
onClick	클릭할 때
onChange	입력값이 바뀔 때
onSubmit	폼이 제출될 때
onmouseenter	마우스가 올라왔을 때
onKeyDown	키를 눌렀을 때

Event 처리

```
type Props = { type: 'date' | 'time' | 'all' };
const current = (type: Props['type']) => {
  const d = new Date();
  switch (type) {
    case 'date':
      console.log(d.toLocaleDateString()); break;
    case 'time':
      console.log(d.toLocaleTimeString()); break;
    default:
      console.log(d.toString());
  }
};
export default function EventBasic({ type }: Props) {
  return (
    <div>
      <button onClick={() => current(type)}>
        현재 시각 가져오기
      </button>
    </div>);
}
```

Component 상태관리

```
interface CounterProps {  
  init: number;  
}  
export default function Counter({init}: CounterProps) {  
  let count = init;  
  const handleClick = () => {  
    // 상태를 변경해야 React가 화면을 다시 그립니다.  
    count += 1;  
  };  
  
  return (  
    <div>  
      <h1>클릭 카운터</h1>  
      <p>현재 클릭 수: {init}</p>  
      <button onClick={handleClick}>클릭하세요</button>  
    </div>  
  );  
}
```

Component 상태관리

```
export default function Counter({ init }: CounterProps) {  
  const [count, setCount] = useState<number>(init);  
  
  const handleClick = (): void => {  
    setCount(count + 1);  
    // 이전 상태값을 안전하게 참조하기 위해 함수형 업데이트를 사용하는 것이 권장됩니다.  
    // setCount((prev) => prev + 1);  
  };  
  
  return (  
    <div style={{ textAlign: 'center', marginTop: '50px' }}>  
      <h1>클릭 카운터</h1>  
      <p>현재 클릭 수: {count}</p>  
      <button onClick={handleClick}>클릭하세요</button>  
    </div>  
  );  
}
```

State란

- State는 컴포넌트의 동적인 데이터를 관리하는 객체
- State는 컴포넌트 내부에서 변경될 수 있으며, 이 상태가 변경되면 리액트는 자동으로 해당 컴포넌트를 다시 렌더링하여 UI를 업데이트
- State의 값을 바꿀때는 setState함수를 사용해서 바꿔야 다시 렌더링 해줌
- 사용자 입력, 버튼 클릭, 네트워크 요청 등의 이벤트에 따라 상태가 변하고, 이에 맞춰 화면이 동적으로 반응

Class Component

```
import { Component } from 'react';
```

```
type Props = {  
  init: number;  
};
```

```
type State = {  
  count: number;  
};
```

```
export default class ClassCounter  
  extends Component<Props, State> {  
  constructor(props: Props) {  
    super(props);  
    this.state = {  
      count: props.init,  
    };  
  }  
}
```

=> 계속

Class Component

=> 앞에서 계속

```
handleClick = () => {  
  this.setState((prevState) => ({  
    count: prevState.count + 1,  
  }));  
};
```

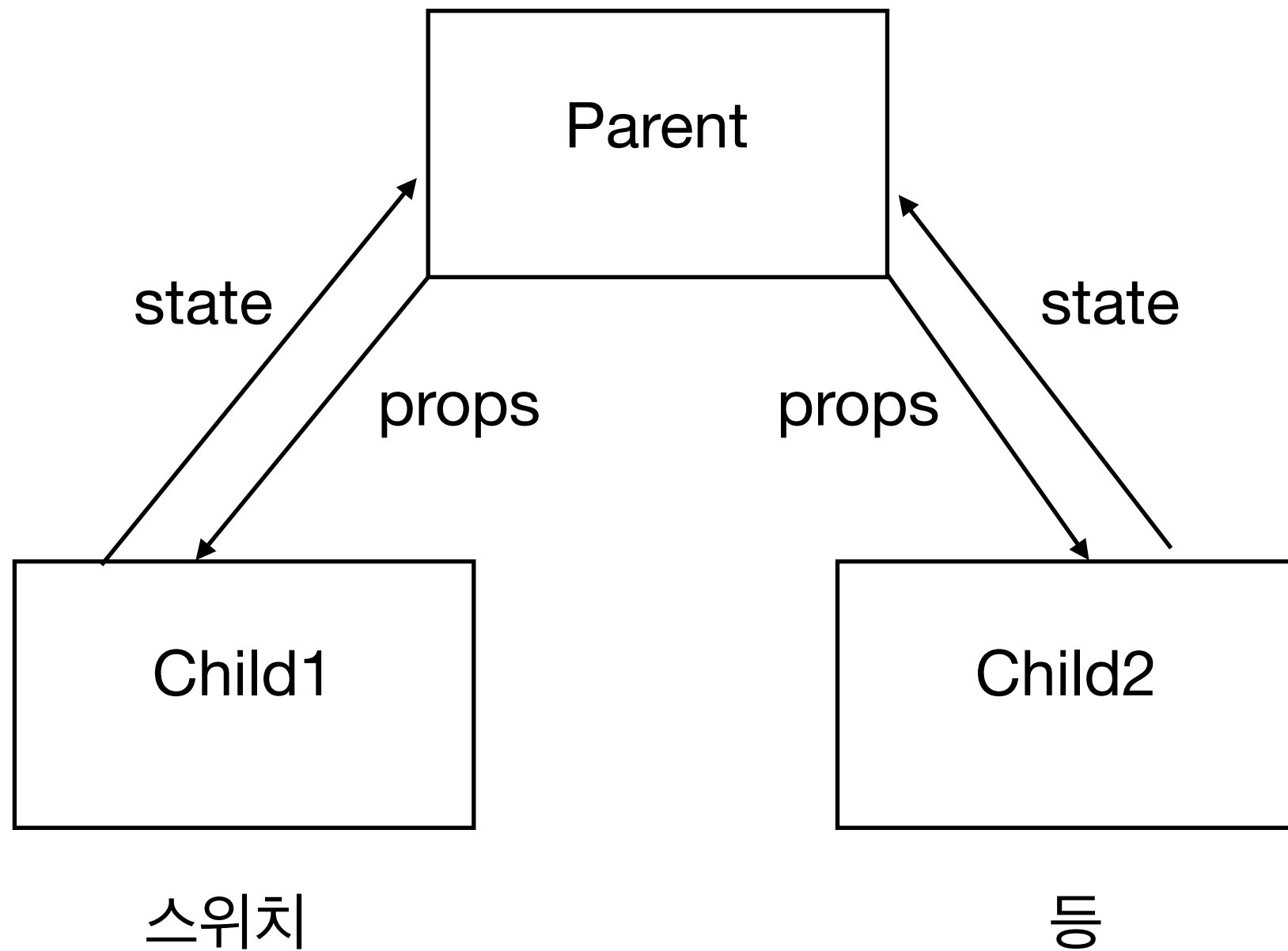
```
render() {  
  return (  
    <div>  
      <h1>클릭 카운터</h1>  
      <p>현재 클릭 수: {this.state.count}</p>  
      <button onClick={this.handleClick}>클릭하세요</button>  
    </div>  
  );  
}
```


useState 활용

```
import { useState } from 'react';
import type { ChangeEvent } from 'react';
export default function LiveInput() {
  const [text, setText] = useState<string>('');
  const handleChange = (e: ChangeEvent<HTMLInputElement>) => {
    setText(e.target.value);
  };
  return (
    <div>
      <h2>✎ 실시간 입력 보기</h2>
      <input
        type="text"
        value={text}
        onChange={handleChange}
        placeholder="내용을 입력하십시오"
      />
      <p>입력한 내용: <strong>{text}</strong></p>
    </div>
  );
}
```

State 관리

State 끌어올리기



State 끌어올리기-Child1

```
interface Child1Props {  
  onChange: (value: string) => void;  
}  
  
export default function Child1({ onChange }: Child1Props) {  
  const handleChange = (event: React.ChangeEvent<HTMLInputElement>) => {  
    onChange(event.target.value);  
  };  
  
  return (  
    <div>  
      <input type="text" onChange={handleChange} placeholder="입력하세요" />  
    </div>  
  );  
}
```

State 끌어올리기-Child2

```
interface Child2Props {  
  value: string;  
}  
  
function Child2({ value }: Child2Props) {  
  return (  
    <div>  
      <p>입력된 값: {value}</p>  
    </div>  
  );  
}  
  
export default Child2;
```

State 끌어올리기-Parent

```
export default function ParentComponent() {  
  const [inputValue, setInputValue] = useState<string>('');  
  
  const handleInputChange = (value: string) => {  
    setInputValue(value);  
  };  
  
  return (  
    <div>  
      <h1>State 올리기</h1>  
      { /* 이제 Child1과 Child2는 정의된 타입에 맞는 Props를 전달받게 됩니다. */ }  
      <Child1 onChange={handleInputChange} />  
      <Child2 value={inputValue} />  
    </div>  
  );  
}
```

State 끌어올리기-Child

```
interface StateCounterProps {  
  step: number;  
  onUpdate: (val: number) => void; // 숫자를 받아서 상태를 업데이트하는 함수  
}
```

```
export default function StateCounter({ step, onUpdate }:  
StateCounterProps) {  
  const handleClick = () => onUpdate(step);  
  
  return (  
    <button className="cnt" onClick={handleClick}>  
      <span>{step}</span>  
    </button>  
  );  
}
```


State 끌어올리기-Parent

```
import { useState } from 'react';
import StateCounter from './StateCounter';

export default function StateParent() {
  const [count, setCount] = useState<number>(0);
  const update = (step: number) => setCount((c) => c + step);

  return (
    <>
      <p>총 개수: {count}</p>
      <StateCounter step={1} onUpdate={update} />
      <StateCounter step={5} onUpdate={update} />
      <StateCounter step={-1} onUpdate={update} />
    </>
  );
}
```


State 끌어올리기 실습



슈가

BTS

← 이전

다음 →



RM

BTS

← 이전

다음 →

State 끌어올리기 실습

```
interface NavigationProps {  
  index: number;  
  onChangeIndex: (newIndex: number) => void;  
}  
  
function Navigation({ index, onChangeIndex }: NavigationProps) {  
  const handleNext = () => {  
    onChangeIndex(index + 1);  
  };  
  
  const handlePrev = () => {  
    onChangeIndex(index - 1);  
  };  
  
  return (  
    <div className="button-container">  
      <button className="nav" onClick={handlePrev}>⬅️이전</button>  
      <button className="nav" onClick={handleNext}>다음➡️</button>  
    </div>  
  );  
}
```

State 끌어올리기 실습

```
interface BTSProps {  
  members: Member[];  
}  
  
export default function BTS({ members }: BTSProps) {  
  const [index, setIndex] = useState<number>(0);  
  const changeIndex = (newIndex: number): void => {  
    const len = members.length;  
    setIndex((newIndex + len) % len);  
  };  
  
  return (  
    <div className="member-container">  
      <MemberCard member={members[index]} />  
      <Navigation index={index} onChangeIndex={changeIndex} />  
    </div>  
  );  
}
```

State 끌어올리기 실습 - BTS.css

```
.member-container {  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
  gap: 1rem;  
}
```

```
.button-container {  
  display: flex;  
  gap: 0.5rem;  
}
```

```
button.nav {  
  padding: 10px 15px;  
  font-size: 1rem;  
  cursor: pointer;  
}
```

조건부 Rendering

조건부 Rendering

```
type GreetingProps = {  
  isLoggedIn: boolean;  
};
```

```
function UserGreeting() {  
  return <h1>다시오셨군요!</h1>;  
}
```

```
function GuestGreeting() {  
  return <h1>회원가입을 해주세요.</h1>;  
}
```

```
function Greeting({ isLoggedIn }: GreetingProps) {  
  return isLoggedIn ? <UserGreeting /> : <GuestGreeting />;  
}
```

조건부 Rendering

```
type ButtonProps = {  
  onClick: () => void;  
};
```

```
function LoginButton({ onClick }: ButtonProps) {  
  return <button onClick={onClick}>로그인</button>;  
}
```

```
function LogoutButton({ onClick }: ButtonProps) {  
  return <button onClick={onClick}>로그아웃</button>;  
}
```


Inline 조건부 Rendering

```
export default function LoginControl() {  
  const [isLoggedIn, setIsLoggedIn] = useState<boolean>(false);  
  const handleLoginClick = () => {  
    setIsLoggedIn(true);  
  };  
  const handleLogoutClick = () => {  
    setIsLoggedIn(false);  
  };  
  return (  
    <div>  
      <Greeting isLoggedIn={isLoggedIn} />  
      {isLoggedIn ? (  
        <LogoutButton onClick={handleLogoutClick} />  
      ) : (  
        <LoginButton onClick={handleLoginClick} />  
      )}  
    </div>  
  );  
}
```


Array Rendering

BTS-App 만들기



BTS



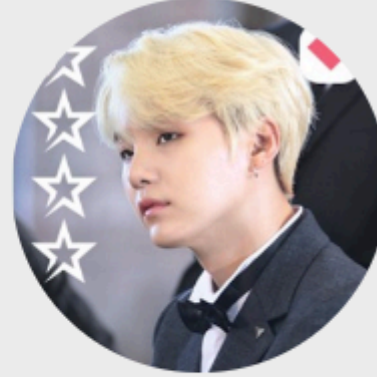
RM

BTS



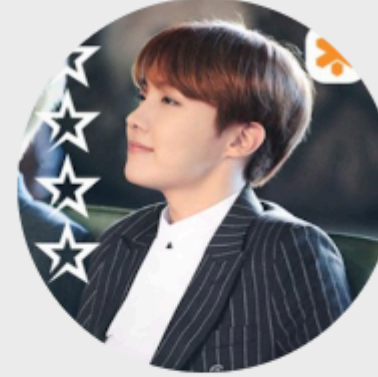
진

BTS



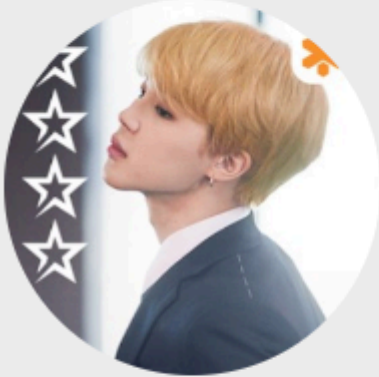
슈가

BTS



제이홉

BTS



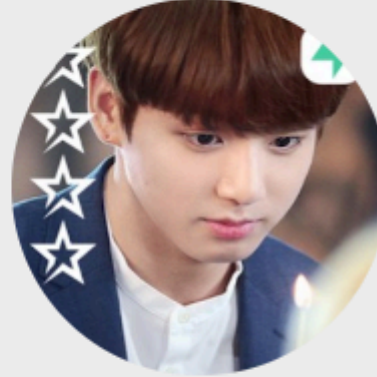
지민

BTS



뷔

BTS



정국

BTS

map함수를 사용한 Array Rendering

```
function App() {  
  return (  
    <>  
      <div>  
        {members.map((member) => (  
          <React.Fragment key={member.nick}>  
            <MemberCard member={member}></MemberCard>  
          </React.Fragment>  
        ))}  
      </div>  
    </>  
  );  
}
```